AI600 – Deep Learning Assignment 3

Namra Nadeem
Roll Number: 25280097
GitHub Repository: `https://github.com/namranadeem-wq/DL-Assignment-3`

Contents

1 Summary of All Tasks **2**

2 Analytical Section **2**
2.1 Q1: Forward and Backward Pass 2
2.2 Q3: Pooling and Normalization 3

3 Programming Section **4**
3.1 Task 1: Custom CNNs and Shortcut Learning 4
3.2 Task 2: Transfer Learning and Interpretability 7

4 Conclusion **10**

1 Summary of All Tasks

This assignment combined analytical understanding of convolutional neural networks with practical deep learning implementation. In the analytical section, the Toy-CNN was studied through explicit forward and backward pass calculations. This developed an understanding of convolution, ReLU activation, gradient flow, pooling, and normalization. In particular, the manual derivations demonstrated how local receptive fields and weight sharing influence both activations and gradients.

In the programming section, the first task required building a compact CNN for standard MNIST under architectural constraints. The model achieved strong performance with closely matched training, validation, and test accuracy, indicating good generalization. The same architecture was then adapted for Colored-MNIST, where the gap between biased and unbiased accuracy revealed shortcut learning. This experiment showed that neural networks can exploit spurious correlations, such as color, instead of learning the true robust feature, which in this case was digit shape.

The second programming task used transfer learning with a pretrained ResNet-18 on STL-10. By freezing the backbone and training only the final classification head, the model achieved strong accuracy efficiently. GradCAM was then used to interpret model decisions. For correct predictions, the heatmaps showed attention on the main object, while for incorrect predictions the attention often shifted to misleading background regions or partial structures. Overall, the assignment illustrated three important ideas: low-level feature learning in CNNs, shortcut learning under dataset bias, and the usefulness of transfer learning and interpretability methods in practical computer vision.

2 Analytical Section

2.1 Q1: Forward and Backward Pass

Q1(a) Forward Pass

The input image and filter are:

$$X = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 2 \end{bmatrix}, \qquad W = \begin{bmatrix} 1 & 0 \\ -1 & 1 \end{bmatrix}$$

Using stride 1 and no padding, the convolution output is:

$$Z = \begin{bmatrix} 2 & 2 \\ -1 & 3 \end{bmatrix}$$

Applying ReLU:

$$A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$$

Flattening gives:

$$A_{\text{flat}} = [2, 2, 0, 3]$$

Using

$$V = [1, -1, 2, 0.5], \qquad c = 1$$

the prediction is:

$$Y = (1)(2) + (-1)(2) + (2)(0) + (0.5)(3) + 1 = 2.5$$

The loss is:

$$L = \frac{1}{2}(Y - T)^2 = \frac{1}{2}(2.5 - 3)^2 = 0.125$$

Q1(b) Backward Pass

First,

$$\frac{\partial L}{\partial Y} = Y - T = -0.5$$

Gradient with respect to fully connected weights:

$$\frac{\partial L}{\partial V} = [-1, -1, 0, -1.5]$$

Gradient with respect to the convolution filter:

$$\frac{\partial L}{\partial W} = \begin{bmatrix} 0.25 & -1.25 \\ -0.5 & -0.5 \end{bmatrix}$$

Q1(c) Interpretation

The weight w_{12} would change the most because it has the largest gradient magnitude:

$$\left| \frac{\partial L}{\partial w_{12}} \right| = 1.25$$

Since its gradient is negative, gradient descent increases this weight. Structurally, this happens because that filter position participates in multiple receptive fields and receives strong contributions from active spatial locations in the forward pass.

2.2 Q3: Pooling and Normalization

Q3(a) Max Pooling

With global max pooling applied directly to

$$A = \begin{bmatrix} 2 & 2 \\ 0 & 3 \end{bmatrix}$$

the output becomes:

$$Y = \max(A) = 3$$

Only the maximum activation receives gradient, so the backward flow becomes sparse. Pixels outside the receptive field of that maximum activation have zero gradient.

Q3(b) Normalization

The mean of the feature map is:

$$\mu = \frac{2+2+0+3}{4} = 1.75$$

The variance is:

$$\sigma^2 = 1.1875$$

Thus, the normalized feature map is:

$$A_{\text{norm}} = \frac{A-\mu}{\sqrt{\sigma^2}}$$

Q3(c) Interpretation

Max pooling provides translation invariance because the output depends only on the maximum activation. If the feature shifts slightly but the same maximum value remains in the pooling region, the output and therefore the loss remain unchanged.

3 Programming Section

3.1 Task 1: Custom CNNs and Shortcut Learning

Part A: Standard MNIST

A custom CNN was implemented for MNIST under the assignment constraints. The final model had 26,698 trainable parameters, satisfying the requirement of staying below 50,000.

The model achieved:

- Training Accuracy: 98.71%
- Validation Accuracy: 98.48%
- Test Accuracy: 98.56%

Analytical Question 1.1 The model generalized well. The training and validation curves were very close, and there was no large gap between them. This indicates that the model learned useful digit features without severe overfitting or underfitting.

Analytical Question 1.2 The first-layer filters captured basic visual primitives such as edges, strokes, and simple intensity transitions. This is expected in CNNs, since early layers typically learn low-level features that later layers combine into more complex digit representations.

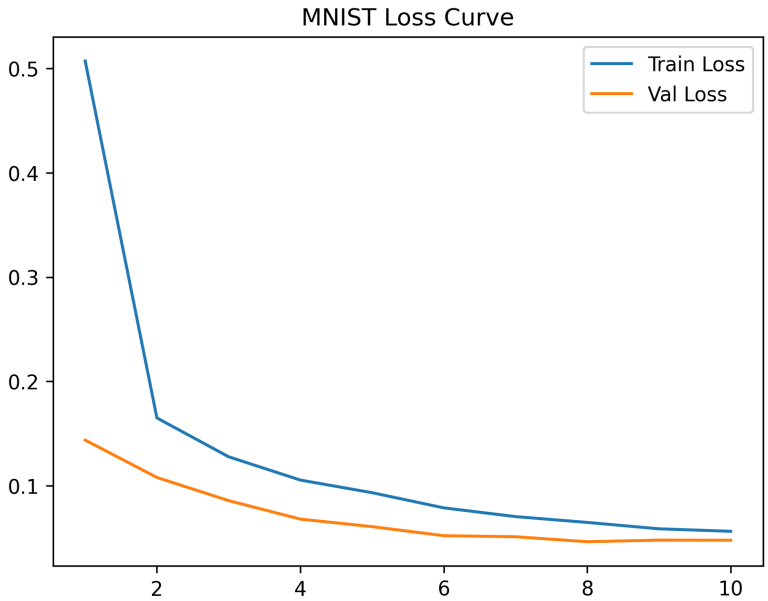

Figure 1: MNIST training and validation loss curves

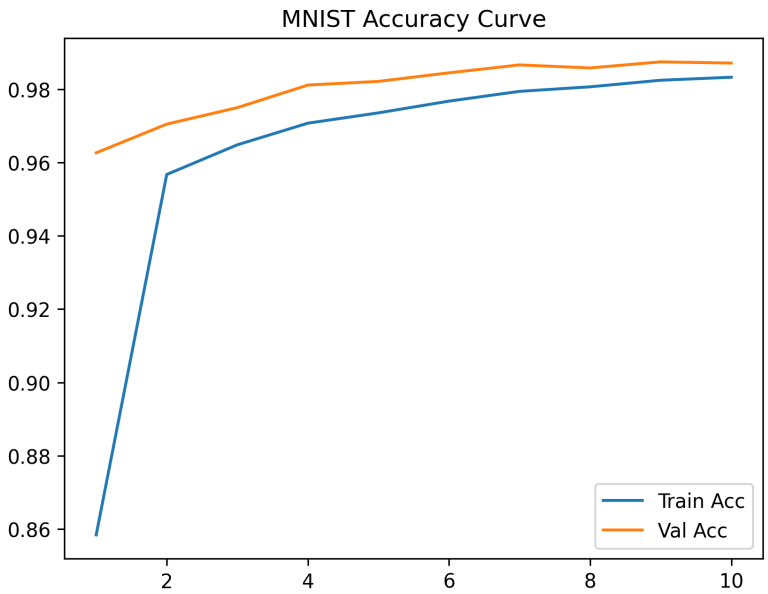

Figure 2: MNIST training and validation accuracy curves

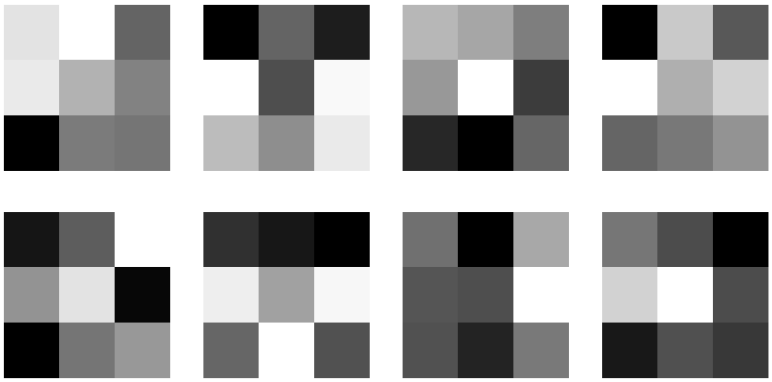

Figure 3: First-layer convolutional filters learned by the MNIST CNN

Part B: Colored-MNIST

The same CNN was adapted to accept RGB input and trained on the Colored-MNIST dataset.

The model achieved:

- Biased Test Accuracy: 98.81%
- Unbiased Test Accuracy: 81.67%

Analytical Question 1.3 The large drop in performance on the unbiased test set shows that the model relied on the spurious correlation between color and label. During optimization, color acted as an easier predictive feature than shape, so gradient descent favored it because it reduced the training loss more quickly.

Analytical Question 1.4 To encourage the model to focus on shape rather than color, one could use grayscale conversion, random color augmentation, better-balanced datasets, or multi-environment training where color is not consistently correlated with label.

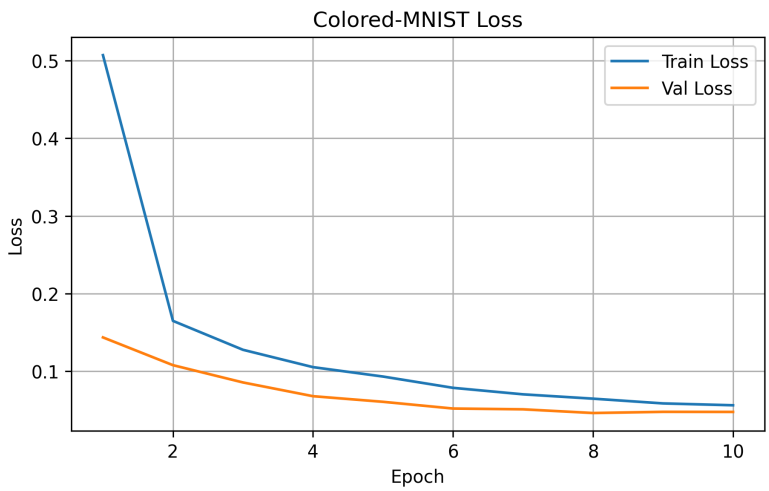

Figure 4: Colored-MNIST training and validation loss curves

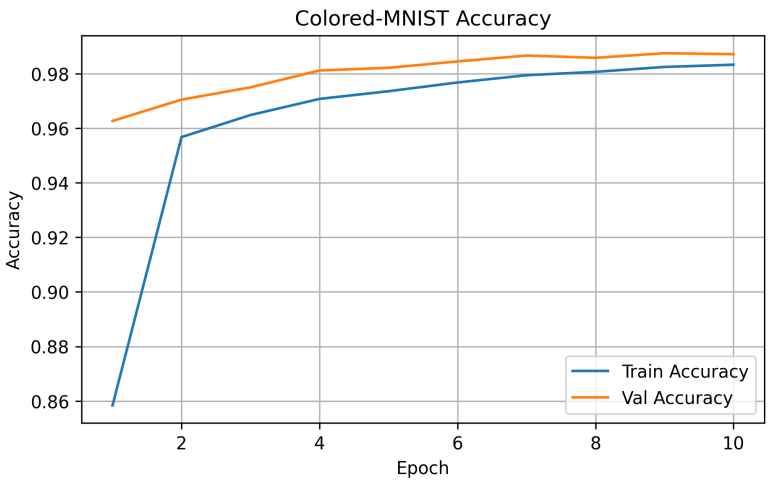

Figure 5: Colored-MNIST training and validation accuracy curves

3.2 Task 2: Transfer Learning and Interpretability

Part A: Fine-Tuning ResNet-18 on STL-10

A pretrained ResNet-18 was loaded from `torchvision.models`. All backbone layers were frozen, and the final fully connected layer was replaced with a new 10-class classifier for STL-10.

The model achieved:

$$\textbf{Test Accuracy} = \mathbf{94.24\%}$$

Analytical Question 2.1 Freezing the early layers is computationally beneficial because it reduces the number of trainable parameters and speeds up training. It is also functionally beneficial because early layers in pretrained networks usually capture general features such as edges, textures, and simple shapes, which transfer well across datasets. Later layers are more task-specific and therefore the final classifier is adapted to the new task.

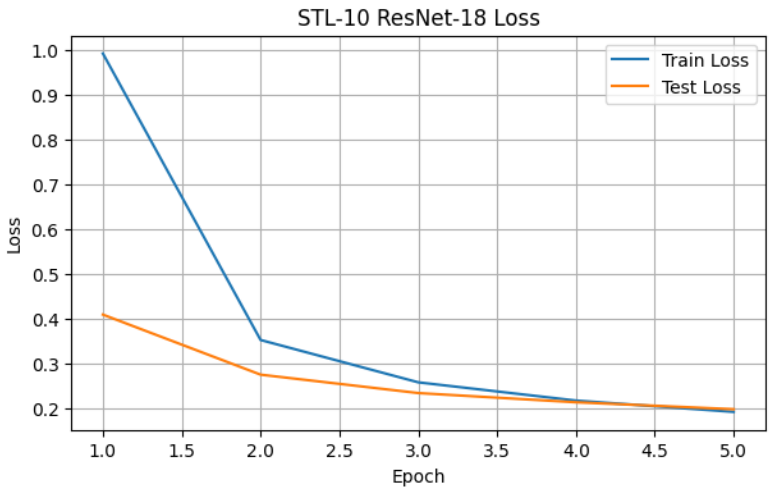

Figure 6: STL-10 ResNet-18 training and test loss curves

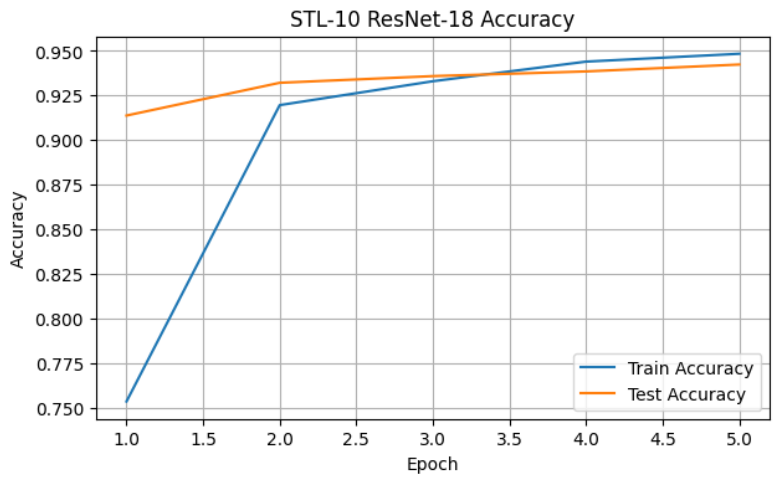

Figure 7: STL-10 ResNet-18 training and test accuracy curves

Part B: Visualizing Decisions with GradCAM

GradCAM was used to inspect which regions of the image contributed most strongly to the model's decision.

Analytical Question 2.2 For correctly classified images, the heatmaps show that the model focuses on the primary object. This suggests that the classifier is using relevant object-level features rather than relying mainly on background context.

Figure 8: GradCAM visualizations for correct predictions

Analytical Question 2.3 For incorrectly classified images, the heatmaps show attention on misleading regions such as background context or only partial parts of the object. This suggests that the model makes mistakes when it relies on incomplete visual evidence or confusing contextual features rather than the full object structure.

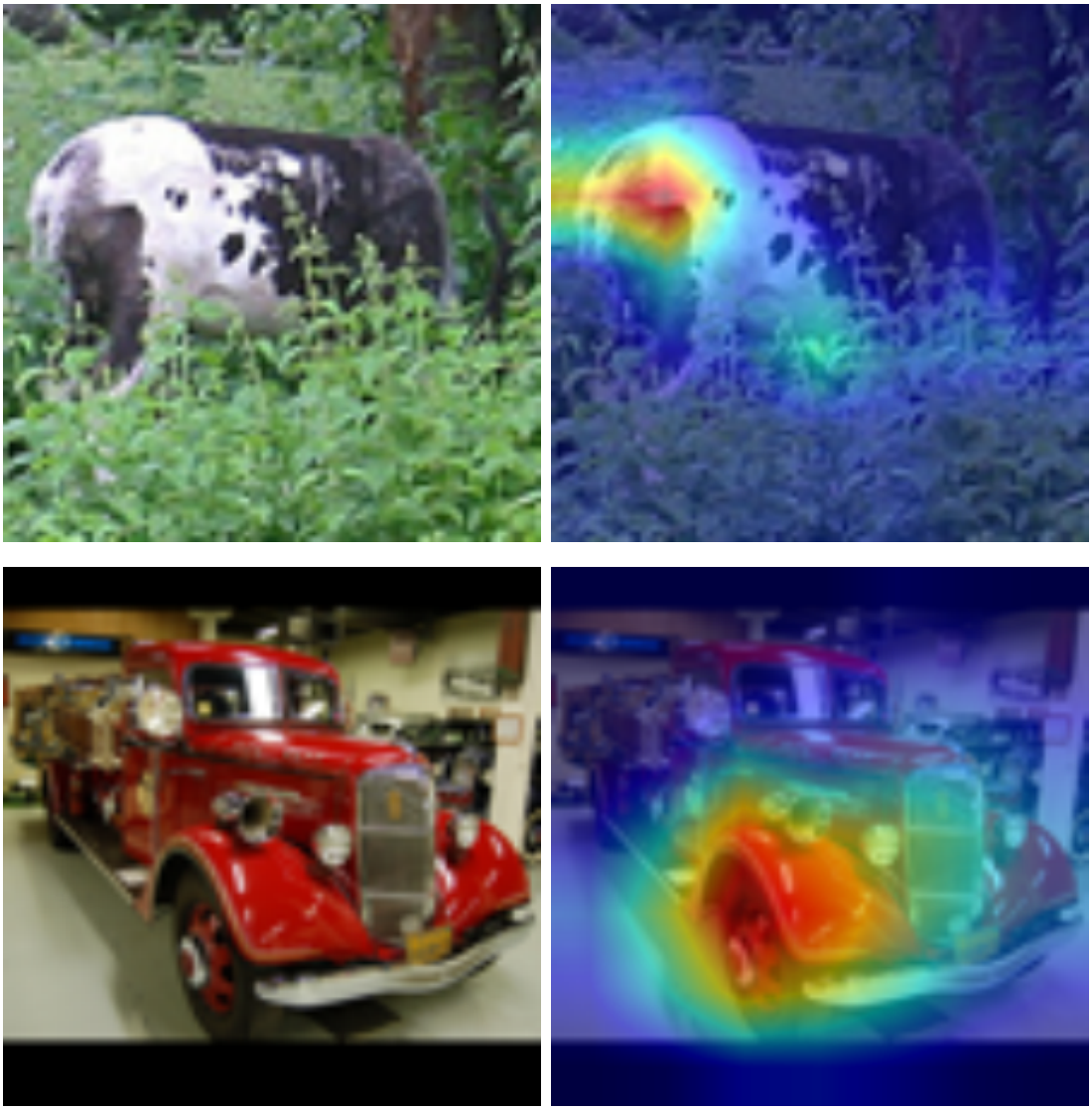

Figure 9: GradCAM visualizations for incorrect predictions

4 Conclusion

This assignment connected mathematical understanding with practical deep learning implementation. The Toy-CNN analysis illustrated the mechanics of forward and backward propagation. The MNIST model showed strong generalization, while the Colored-MNIST experiment revealed shortcut learning through spurious correlations. Finally, transfer learning with ResNet-18 achieved strong performance on STL-10, and GradCAM provided interpretability into both successful and failed predictions.

Generative AI Usage

Generative AI was used as a learning and writing aid for:

- understanding the analytical derivations,
- debugging PyTorch code,

- interpreting training curves and GradCAM outputs,
- organizing the report in LaTeX.

Example prompts included:

- "Explain the Toy-CNN forward and backward pass step by step."
- "Give PyTorch code for a CNN under 50k parameters for MNIST."
- "Help interpret GradCAM outputs for correct and incorrect predictions."
- "Generate LaTeX report sections in assignment order."

The outputs were reviewed and edited before inclusion in the final report.